

Homework 2 - Sorting

Name: 임도협 (Dohyeop Lim)

StudentID: 24101518

[1. Insertion Sort]

Implement `insertion_sort(...)` in `sort.c`. The function takes three arguments: an array of integers, `low` and `high` indices to denote the range to be sorted. Thus, the function sorts the array of range `[low, high)`. The function must produce the array of integers, whose elements are located in ascending order.

```
1 void insertion_sort(int arr[], int low, int high)
2 {
3     for (int i = low + 1; i < high; i++) {
4         int key = arr[i];
5         int j = i - 1;
6         while (j >= low && arr[j] > key) {
7             arr[j + 1] = arr[j];
8             j--;
9         }
10        arr[j + 1] = key;
11    }
12 }
```

[2. Merge Sort]

- A. Implement `merge(...)` in `sort.c`. The function takes four arguments: an array of integers, `low`, `mid`, and `high` indices. The `low` and `high` indices denote the range to be sorted. The `mid` is defined by $(low + high) / 2$. Therefore, the `low` and the `mid` denote the start indices of the left and the right subarrays, respectively. The output of the function must be the sorted array of size `high - low`.

```
1 void merge(int arr[], int low, int mid, int high)
2 {
3     int n1 = mid - low;
4     int n2 = high - mid;
5
6     int* left = (int*)malloc(sizeof(int) * n1);
7     int* right = (int*)malloc(sizeof(int) * n2);
8
9     for (int i = 0; i < n1; i++)
10        left[i] = arr[low + i];
11    for (int j = 0; j < n2; j++)
12        right[j] = arr[mid + j];
13
14    int i = 0, j = 0, k = low;
```

```

15     while (i < n1 && j < n2) {
16         if (left[i] <= right[j]) {
17             arr[k++] = left[i++];
18         } else {
19             arr[k++] = right[j++];
20         }
21     }
22     while (i < n1)
23         arr[k++] = left[i++];
24     while (j < n2)
25         arr[k++] = right[j++];
26
27     free(left);
28     free(right);
29 }

```

B. Implement `merge_sort(...)` in `sort.c`.

```

1 void merge_sort(int arr[], int low, int high)
2 {
3     if (high - low <= 1) return;
4     int mid = low + (high - low) / 2;
5     merge_sort(arr, low, mid);
6     merge_sort(arr, mid, high);
7     merge(arr, low, mid, high);
8 }

```

[3. Quick Sort]

A. Implement `partition(...)` in `sort.c`. The function takes three arguments: an array of integers, `low`, and `high` indices. The `low` and `high` indices denote the range to be partitioned. The function outputs the index of pivot in the partitioned array. For the partitioning, you should select random pivot. Also, you can use `swap()` in `util.h`

```

1 int partition(int arr[], int low, int high)
2 {
3     int pivot = arr[high - 1];
4     int i = low - 1;
5     for (int j = low; j < high - 1; j++) {
6         if (arr[j] <= pivot) {
7             i++;
8             swap(&arr[i], &arr[j]);
9         }
10    }
11    swap(&arr[i + 1], &arr[high - 1]);
12    return i + 1;
13 }

```

B. Implement `quick_sort(...)` in `sort.c`.

```

1 void quick_sort(int arr[], int low, int high)
2 {
3     if (low < high - 1) {
4         int p = partition(arr, low, high);
5         quick_sort(arr, low, p);
6         quick_sort(arr, p + 1, high);
7     }
8 }

```

[4. Comparison of Sorting Algorithms]

- A. Explain the difference between three types of input arrays in main.c: `random_array`, `partial_random_array1`, `partial_random_array2`.

`random_array` is fully random with no order.
`partial_random_array1` is mostly sorted but has a random part mixed in (array is mostly sorted except the tail).
`partial_random_array2` is even closer to sorted, with a smaller random part (almost sorted with only scattered disorder).

- B. Measure the execution time of three algorithms using input arrays of different size: 10, 30, 50, 100, 500, 1000, 10000, 100000, 1000000, 10000000. Plot graph to compare the execution time of three algorithm for each type of random array.

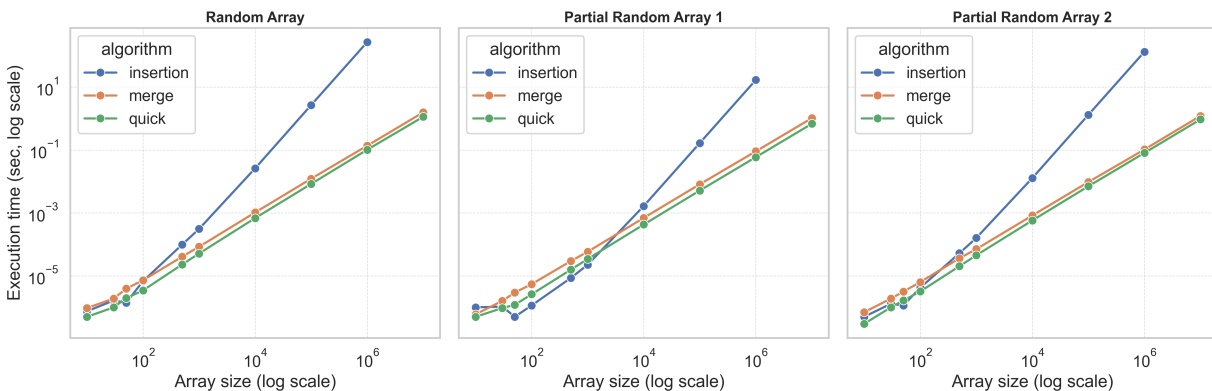


Figure 1: Execution time comparison of insertion sort¹, merge sort, and quick sort on three input types. Both axes are shown in log scale. The complete benchmark table is available in Appendix 1.

- C. Let's consider `random_array` data. Which algorithm is the best when the array size is small (e.g., 10, 30, 50). Explain why.

Insertion sort

For small arrays, the overhead of recursion in merge & quick sort is bigger than the actual work.

Insertion sort is simple and runs very fast on small inputs, so it wins here!

- D. Let's consider `random_array` data. Which algorithm becomes the eventually winner? Explain why.

Quick sort

From the results, at $n = 1,000,000$ quick sort takes about 0.10 sec, while merge sort takes 0.13 sec and insertion sort takes 267 sec.

At $n = 10,000,000$ quick sort still leads (1.13 sec) over merge sort (1.57 sec).

Thus, quick sort becomes the winner for large random arrays!

- E. Let's consider the insertion sort algorithm. Which types of input arrays does insert sort have an advantage over other algorithms? Explain why.

Partial random arrays (1 & 2)

For $n = 1,000,000$, insertion sort on random arrays took 267 sec, but on partial random arrays it only took 16.7 sec and 131 sec.

This shows insertion sort benefits when the data is already mostly sorted, since it only needs a small number of shifts.

- F. Explain why quick sort is faster than merge sort.

a

- G. Explain the advantages that merge sort has over quick sort.

a

[5. Faster Merge Sort]

- A. Implement the fast merge sort algorithm. The improved version divides the input array only until the size of the subarrays becomes less than 32. It then perform the insertion sort to conquer these small subarrays.

```
1 #define THRESHOLD 32
2
3 void merge_sort_2(int arr[], int low, int high)
4 {
5     if (high - low <= THRESHOLD) {
6         insertion_sort(arr, low, high);
7         return;
8     }
```

```

9
10     int mid = low + (high - low) / 2;
11     merge_sort_2(arr, low, mid);
12     merge_sort_2(arr, mid, high);
13     merge(arr, low, mid, high);
14 }

```

- B. Compare the fast merge sort with its original one on the settings of problem 4.B. Is performance improvement observable? Explain why it is faster.

Yes

In the original version, every subarray is divided down to size 1, so it creates deep recursion calls.

However in the faster version, once it reaches the threshold, insertion sort is used instead of further splitting. Since insertion sort runs very efficiently on small and nearly sorted subarrays, this reduces the recursion overhead!

They both take $O(n \log n)$ time, but the new one is faster in practice!

- C. Implement the faster merge sort algorithm by modifying the fast merge sort. This version first determine the necessity of merge operation before merging, and then skips the merge operation, when it is unnecessary.

```

1  #define THRESHOLD 32
2
3  void merge_sort_3(int arr[], int low, int high)
4  {
5      if (high - low <= THRESHOLD) {
6          insertion_sort(arr, low, high);
7          return;
8      }
9
10     int mid = low + (high - low) / 2;
11     faster_merge_sort(arr, low, mid);
12     faster_merge_sort(arr, mid, high);
13
14     if (arr[mid - 1] > arr[mid]) {
15         merge(arr, low, mid, high);
16     }
17 }

```

- D. For which types of input array will this version produce performance improvements?. Explain why.

Partial random arrays (1 and 2). Since large portions are already sorted, many merges can be skipped.

Appendix

Algorithm	Size	Random Array	Partial Random 1	Partial Random 2
Insertion	10	0.00000075	0.00000100	0.00000050
Insertion	30	0.00000165	0.00000105	0.00000130
Insertion	50	0.00000140	0.00000050	0.00000115
Insertion	100	0.00000680	0.00000115	0.00000420
Insertion	500	0.00009885	0.00000855	0.00005225
Insertion	1,000	0.00030985	0.00002265	0.00016035
Insertion	10,000	0.02607045	0.00165270	0.01282990
Insertion	100,000	2.67055610	0.16602390	1.30848705
Insertion	1,000,000	267.85176080	16.72882800	131.53064900
Merge	10	0.00000095	0.00000060	0.00000070
Merge	30	0.00000190	0.00000160	0.00000190
Merge	50	0.00000390	0.00000295	0.00000315
Merge	100	0.00000725	0.00000545	0.00000635
Merge	500	0.00004080	0.00002940	0.00003615
Merge	1,000	0.00008410	0.00005845	0.00007080
Merge	10,000	0.00103160	0.00069455	0.00082880
Merge	100,000	0.01220060	0.00812765	0.00955915
Merge	1,000,000	0.13641800	0.09043000	0.10348600
Merge	10,000,000	1.56927100	1.02761500	1.20692300
Quick	10	0.00000050	0.00000050	0.00000030
Quick	30	0.00000100	0.00000095	0.00000100
Quick	50	0.00000195	0.00000120	0.00000165
Quick	100	0.00000345	0.00000260	0.00000320
Quick	500	0.00002310	0.00001570	0.00002020
Quick	1,000	0.00005120	0.00003435	0.00004505
Quick	10,000	0.00068425	0.00042760	0.00057585
Quick	100,000	0.00833690	0.00513570	0.00702175
Quick	1,000,000	0.10170800	0.05907400	0.08065700
Quick	10,000,000	1.13447600	0.68083100	0.93708000

Table 1: Full benchmark results: average execution time (sec) of sorting algorithms on different array sizes and input types.